

Numéro d'équipe: #102

Noms, prénoms et matricules:

- Machraoui, Simon : 2487824
- Beaumier, Édouard : 2087482

TP5 - Échantillonnage

```
In [20]: import numpy as np
import matplotlib.pyplot as plt
from numpy.fft import fft, fftfreq
from scipy.signal import resample
from scipy.interpolate import interp1d
```

Exercice 2, devoir (4pt/20):

Théorème de reconstruction de Shannon

La capacité de reconstruire un signal continu à partir de ses échantillons est fondamentale en traitement du signal. Selon le théorème de reconstruction de Shannon, un signal continu peut être parfaitement reconstruit à partir de ses échantillons s'il est limité en bande et échantillonné à une fréquence au moins deux fois supérieure à sa fréquence maximale (fréquence de Nyquist). Une reconstruction précise permet de garantir que l'information d'origine est préservée et peut être utilisée de manière fiable dans diverses applications.

a) [Code] Dans le contexte d'un robot, considérer que les mouvements d'une articulation robotique sont échantillonnés. Utiliser une fonction de reconstruction pour recréer ce signal continu à partir des échantillons d'un signal sinusoïdal de 75 Hz échantillonné à une fréquence de 200 Hz. Afficher le signal original, le signal échantillonné à 200 Hz et le signal reconstruit. **(2pt/20)**

```
In [25]: def sinc_interpolation(t, t_sampled, x_sampled):
    T = t_sampled[1] - t_sampled[0] # Intervalle ech. = 1/fs
    reconstructed = np.zeros_like(t)
```

```

for n, tn in enumerate(t_sampled):
    reconstructed += x_sampled[n] * np.sinc((t - tn) / T)

return reconstructed

```

In [26]: # Écrire votre code ici pour a)

```

# --- Données ---
f_signal = 75          # fréquence du signal (Hz)
fs = 200              # fréquence d'échantillonnage (Hz)
t1, t2 = 0, 0.1       # intervalle de temps (secondes)

# --- Génération du signal "continu" (référence) ---
fs_ref = 10000         # fréquence d'échantillonnage très élevée pour
t_ref = generate_time_vec(t1, t2, fs_ref, endpoint=False)
x_ref = np.sin(2*np.pi*f_signal*t_ref)

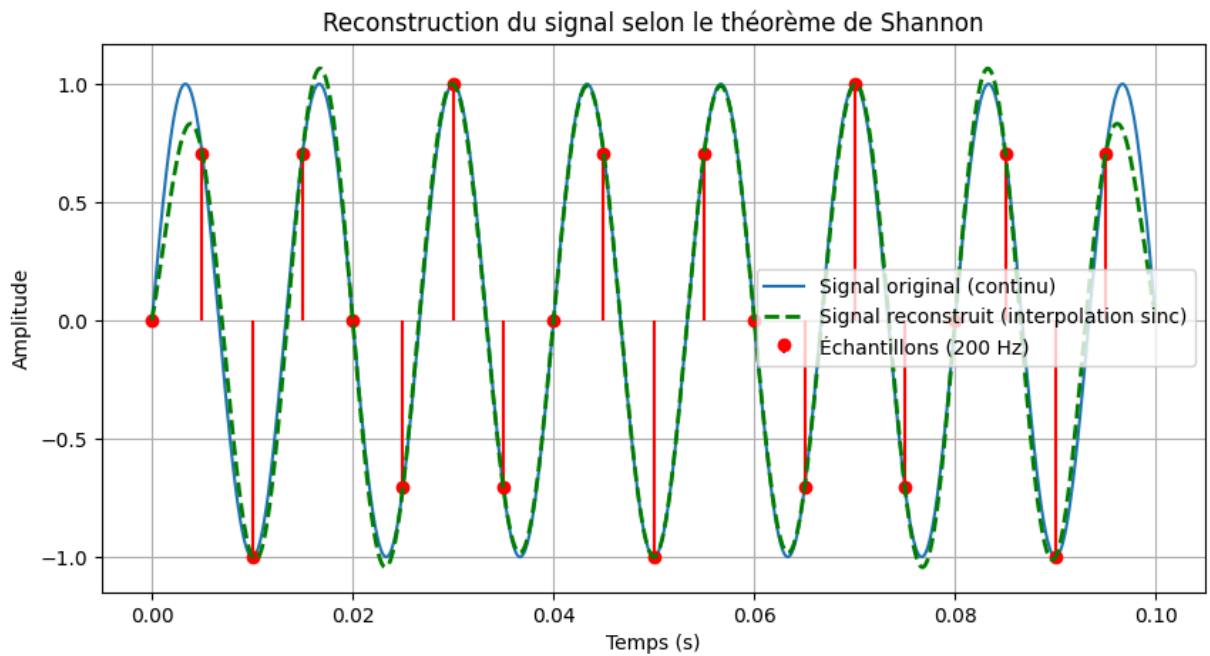
# --- Signal échantillonné ---
t_sampled = generate_time_vec(t1, t2, fs, endpoint=False)
x_sampled = np.sin(2*np.pi*f_signal*t_sampled)

x_reconstructed = sinc_interpolation(t_ref, t_sampled, x_sampled)

# --- Tracé des signaux ---
plt.figure(figsize=(10,5))
plt.plot(t_ref, x_ref, label="Signal original (continu)", linewidth=1.5)
plt.stem(t_sampled, x_sampled, linefmt='r-', markerfmt='ro', basefmt=" ", label="Éc
plt.plot(t_ref, x_reconstructed, 'g--', linewidth=2, label="Signal reconstruit (int

plt.title("Reconstruction du signal selon le théorème de Shannon")
plt.xlabel("Temps (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()

```



b) [Analyse écrite] En se basant sur le mouvement de l'articulation robotique, comparer le signal reconstruit avec le signal original. Est-il possible de récupérer fidèlement le signal original à partir des échantillons et de garantir un contrôle précis du robot? **(1.5pt/20)**

Note : Utiliser l'argument `endpoint = False` de la fonction `generate_time_vec` pour un meilleur affichage.

Écrire votre réponse ici pour b).

Le signal reconstruit par interpolation sinc correspond très bien au signal original au centre de l'intervalle (autour de $t \approx 0,05$ s), ce qui confirme que la condition de Shannon est respectée puisque la fréquence d'échantillonnage utilisée ($f_s = 200$ Hz) est bien supérieure à $2 \times f_{\text{signal}} = 150$ Hz. Dans cette zone centrale, la reconstruction est pratiquement parfaite.

Cependant, on observe de légers écarts aux extrémités de l'intervalle (près de $t = 0$ et $t = 0,1$ s). Ces différences proviennent du fait que l'interpolation sinc idéale nécessite une somme infinie d'échantillons : or, dans la pratique, le signal est fini et le noyau sinc est tronqué, ce qui provoque des effets de bord. Cela est normal et n'indique pas une violation du théorème de Shannon.

Pour un mouvement d'articulation robotique, cela signifie que les positions peuvent être reconstruites fidèlement dans la partie active du mouvement, ce qui permet un contrôle précis. Les légers écarts aux bords sont généralement négligeables, ou peuvent être réduits par des techniques pratiques comme l'augmentation du nombre d'échantillons.

Ainsi, oui : il est possible de récupérer quasiment parfaitement le signal original et d'assurer un contrôle précis du robot, tant que l'on échantillonne au-dessus de la fréquence de Nyquist et que l'on tient compte des effets de bord.

c) [Analyse écrite] Donner un désavantage de ce type de reconstruction du signal continu. Justifier dûment. **(0.5pt/20)**

Écrire votre réponse ici pour c).

Un inconvénient de la reconstruction par interpolation sinc est qu'elle est **coûteuse en calcul**,

car elle nécessite une **somme infinie de fonctions sinc** pour chaque point reconstruit.

De plus, la sinc n'est **pas à support fini** (elle s'étend sur tout l'axe du temps),

ce qui rend cette méthode **impraticable en temps réel** pour un système de commande de robot.

Exercice 3, devoir (8pt/20):

Chevauchement et filtres anti-chevauchement

Dans le domaine de la robotique, obtenir des mesures propres est crucial pour le bon fonctionnement des commandes. Cependant, le chevauchement (*aliasing*) est un phénomène indésirable qui peut se produire lors du sous-échantillonnage d'un signal. Un

filtre anti-chevauchement est souvent utilisé pour éviter ce phénomène, garantissant que les capteurs ne fournissent que les informations pertinentes pour le mouvement du robot.

a) [Photo dans case texte] Considérer le signal suivant:

$$x(t) = \sum_{n \in \mathbb{Z}} \tilde{x}(t - nT),$$

où,

$$\tilde{x}(t) = \begin{cases} 1 - \frac{8}{T}t, & \text{si } 0 \leq t < \frac{T}{4} \\ -1, & \text{si } \frac{T}{4} \leq t < \frac{T}{2} \\ -5 + \frac{8}{T}t, & \text{si } \frac{T}{2} \leq t < \frac{3T}{4} \\ 1, & \text{si } \frac{3T}{4} \leq t < T \\ 0, & \text{sinon,} \end{cases}$$

où T est égale à 10 millisecondes. Déterminer analytiquement les fréquences observées si le signal est échantillonné à 450 Hz. **(2pt/20)**

Insérer une photo de vos démarches pour (a) (papier, L^AT_EX ou tablette).

Période : $T = 10 \text{ ms} = 0,01 \text{ s}$

TP5

fréquence fondamentale : $f_0 = \frac{1}{T} = \frac{1}{0,01} = 100 \text{ Hz}$

fréquence des harmoniques : $f_k = f_0 \cdot k$

fréquence d'échantillonnage : $f_s = 450 \text{ Hz} \Rightarrow \frac{f_s}{2} = 225 \text{ Hz}$

d'après le théorème de l'échantillonnage :

un composant continu en fréquence f_k est observé après échantillonnage à la fréquence repliée dans la bande de Nyquist $[0, \frac{f_s}{2}]$: on peut prendre la valeur résiduelle $f_k \bmod f_s$ puis la plier si nécessaire (si $> f_s/2$ on prend $f_s - (f_k \bmod f_s)$).

Calculons les résidus pour $f_k = k \times 100$:

$\hookrightarrow k=0 \Rightarrow f_0 = 0 \Rightarrow$ observable 0 Hz

$\hookrightarrow k=1 \Rightarrow f_1 = 100 \text{ Hz} \Rightarrow$ observable 100 Hz

$\hookrightarrow k=2 \Rightarrow f_2 = 200 \text{ Hz} \Rightarrow$ observable 200 Hz

$\hookrightarrow k=3 \Rightarrow f_3 = 300 \text{ Hz} \Rightarrow 450 - 300 = 150 \text{ Hz} \Rightarrow$ replié en 150 Hz

$\hookrightarrow k=4 \Rightarrow f_4 = 400 \text{ Hz} \Rightarrow 450 - 400 = 50 \text{ Hz} \Rightarrow$ replié en 50 Hz

$\hookrightarrow k=5 \Rightarrow f_5 = 500 \text{ Hz} \Rightarrow 500 \bmod 450 = 50 \text{ Hz} \Rightarrow 50 \text{ Hz}$ même que pour $k=4$.

$\hookrightarrow k=6 \Rightarrow f_6 = 600 \text{ Hz} \Rightarrow 600 \bmod 450 = 150 \text{ Hz} \Rightarrow 150 \text{ Hz}$ même que pour $k=3$.

la suite répète donc les mêmes valeurs.

on a donc un ensemble de valeurs observées dans $[0, 225]$:

$\{0, 50, 100, 150, 200\} \text{ Hz}$

b) [Code] Générer numériquement le signal $x(t)$ et afficher son spectre. Échantillonner à 450 Hz pour valider vos résultats. (1pt/20)

In [27]: # Écrire votre code ici pour b)

```
# --- Paramètres ---
T = 0.01                # 10 ms
f0 = 1/T                # 100 Hz
fs_cont = 20000         # fréquence "quasi-continue" de référence (affichage)
t1, t2 = 0, 100*T      # afficher 5 périodes
```

```

# --- Définition de  $\tilde{x}(t)$  ---
def x_tilde(t, T):
    t_mod = np.mod(t, T)
    x = np.zeros_like(t_mod)

    #  $0 \leq t < T/4$ 
    mask1 = (t_mod >= 0) & (t_mod < T/4)
    x[mask1] = 1 - (8/T)*t_mod[mask1]

    #  $T/4 \leq t < T/2$ 
    mask2 = (t_mod >= T/4) & (t_mod < T/2)
    x[mask2] = -1

    #  $T/2 \leq t < 3T/4$ 
    mask3 = (t_mod >= T/2) & (t_mod < 3*T/4)
    x[mask3] = -5 + (8/T)*t_mod[mask3]

    #  $3T/4 \leq t < T$ 
    mask4 = (t_mod >= 3*T/4) & (t_mod < T)
    x[mask4] = 1

    return x

# --- Liste des fréquences d'échantillonnage pour comparaison ---
fs_list = [fs_cont, 450] # référence + échantillonné
labels = ["Référence haute résolution", "Échantillonné à 450 Hz"]

fig, axes = plt.subplots(len(fs_list), 1, figsize=(10, 8), sharex=False)

# --- Boucle d'affichage des spectres ---
for i, (ax, fs, lab) in enumerate(zip(axes, fs_list, labels)):

    # Signal échantillonné (endpoint=False pour un spectre propre)
    t = generate_time_vec(t1, t2, fs, endpoint=False)
    x = x_tilde(t, T)
    N = len(x)

    # FFT brute (sans fenêtre)
    X = fft(x)
    freqs = fftfreq(N, d=1/fs)
    Xmag = np.abs(X)

    # --- Affichage du spectre miroir ---
    ax.stem(freqs, Xmag, basefmt=" ")
    if i == 0:
        ax.set_xlim(-500, 500) # premier graphe
    else:
        ax.set_xlim(-250, 250) # deuxième graphe
    ax.set_ylabel("|X(f)|")
    ax.set_title(f"Spectre du signal - {lab}")
    ax.grid(True)

    # Tracer Les vraies harmoniques (100, 200, 300, ...)
    for k in range(1, 10):
        f_h = k * f0

```

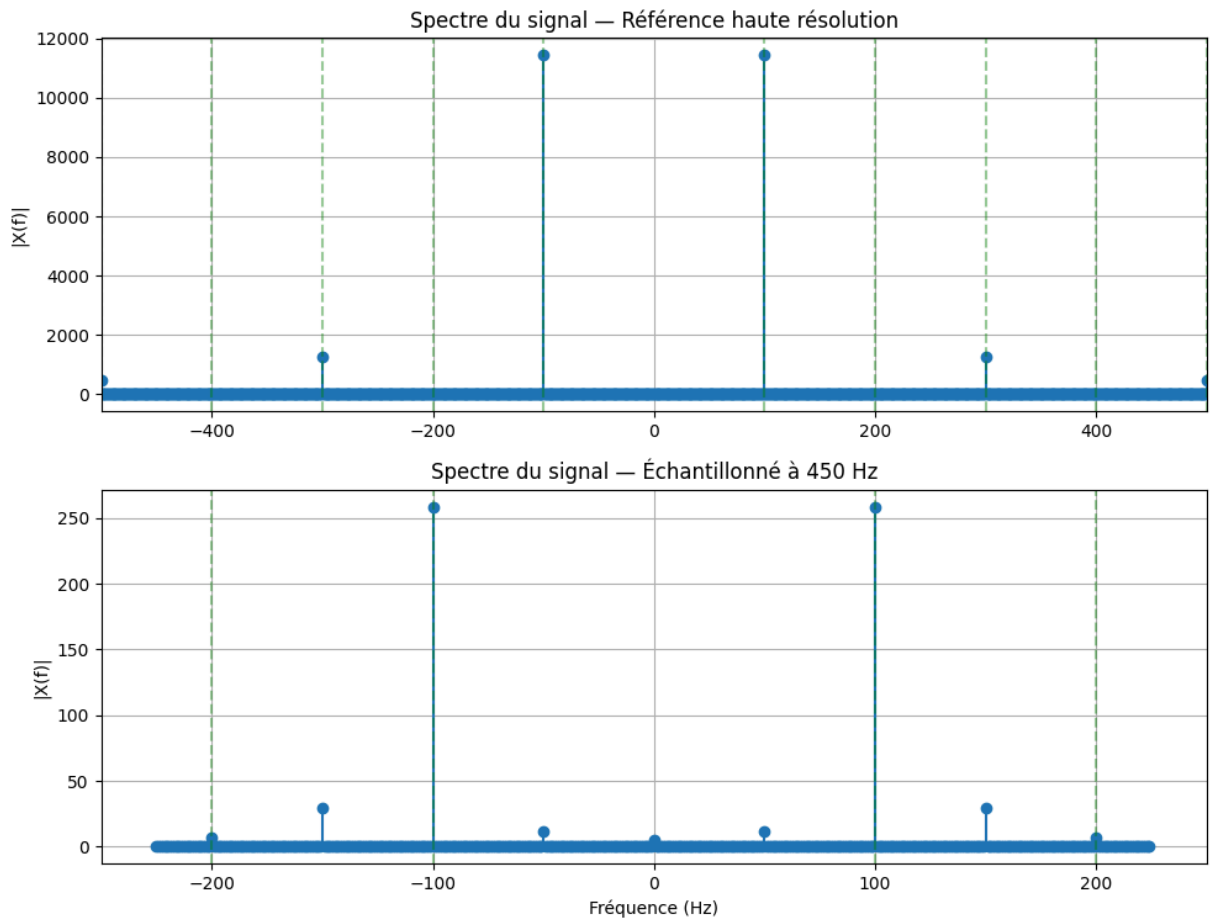
```

ax.axvline(f_h, color='g', linestyle='--', alpha=0.4)
ax.axvline(-f_h, color='g', linestyle='--', alpha=0.4)

axes[-1].set_xlabel("Fréquence (Hz)")
fig.suptitle("Spectres du signal x(t) – FFT (endpoint=False)", y=1.02)
plt.tight_layout()
plt.show()

```

Spectres du signal x(t) — FFT (endpoint=False)



c) [Code] Considérons qu'un capteur du robot mesure le défini en a) avec une fréquence d'échantillonnage de 1200 Hz. Cependant, il y a également des interférences et un signal sinusoïdal parasite est présent à 750 Hz due à la vibration du système. Générer un signal représentant ces mesures, puis l'échantillonner à 1200 Hz. Afficher le spectre du signal échantillonné.

Le signal peut être échantillonné à une nouvelle fréquence d'échantillonnage en utilisant la fonction `np.interp`, qui effectue une interpolation linéaire. Cela permet de reconstruire le signal à des instants de temps spécifiques. (1pt/20)

Note : Utiliser l'argument `endpoint = False` de la fonction `generate_time_vec` pour un meilleur affichage.

In [28]: # Écrire votre code ici pour c)

```
# --- Paramètres ---
T = 0.01                # période du signal périodique (10 ms)
f0 = 1.0 / T            # 100 Hz
t1, t2 = 0.0, 100*T    # durée: 5 périodes pour une bonne résolution spectrale

# capteur / échantillonnage
fs_sensor = 1200        # fréquence d'échantillonnage du capteur (Hz)
fs_cont = 20000         # fréquence "continue" haute résolution pour référence

# parasite
f_parasite = 750.0      # Hz (vibration)
A_parasite = 0.8        # amplitude du parasite

# --- signal "continu" haute résolution ---
t_cont = generate_time_vec(t1, t2, fs_cont, endpoint=False)
x_cont = x_tilde(t_cont, T) + A_parasite * np.sin(2*np.pi*f_parasite*t_cont)

# --- échantillonnage effectué par le capteur (on interpole depuis t_cont) ---
t_s = generate_time_vec(t1, t2, fs_sensor, endpoint=False)
# interpolation linéaire pour obtenir les valeurs aux instants d'échantillonnage
x_s = np.interp(t_s, t_cont, x_cont)

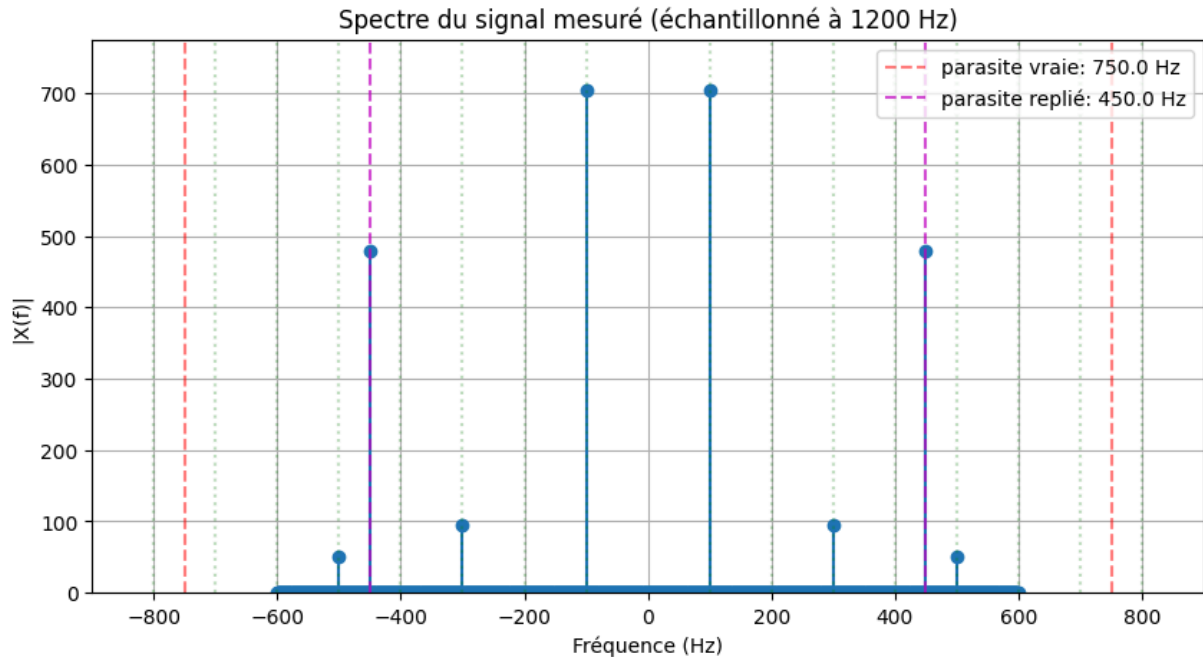
# --- FFT du signal échantillonné ---
N = len(x_s)
X = fft(x_s)
freqs = fftfreq(N, d=1.0/fs_sensor)
Xmag = np.abs(X)

# --- Affichage spectre miroir (positif & négatif) ---
plt.figure(figsize=(10,5))
plt.stem(freqs, Xmag, basefmt=" ")
plt.xlim(-900, 900)
plt.ylim(0, Xmag.max()*1.1)
plt.xlabel("Fréquence (Hz)")
plt.ylabel("|X(f)|")
plt.title("Spectre du signal mesuré (échantillonné à {fs_sensor} Hz)")
plt.grid(True)

# Indiquer la fréquence parasite originale et sa fréquence observée (alias)
# f_parasite = 750 Hz ; avec fs=1200 -> Nyquist fs/2 = 600 Hz -> repli à 1200-750=450 Hz
f_alias = abs(fs_sensor - f_parasite) if f_parasite > fs_sensor/2 else f_parasite
plt.axvline(f_parasite, color='r', linestyle='--', alpha=0.5, label=f'parasite vrai')
plt.axvline(-f_parasite, color='r', linestyle='--', alpha=0.5)
plt.axvline(f_alias, color='m', linestyle='--', alpha=0.7, label=f'parasite replié')
plt.axvline(-f_alias, color='m', linestyle='--', alpha=0.7)

# tracer aussi quelques harmoniques du signal périodique (100,200,...)
for k in range(1, 9):
    axf = k * f0
    plt.axvline(axf, color='g', linestyle=':', alpha=0.25)
    plt.axvline(-axf, color='g', linestyle=':', alpha=0.25)
```

```
plt.legend(loc='upper right')
plt.show()
```



d) [Analyse écrite] Sur la base du signal échantillonné, quelles fréquences observez-vous? Pourquoi ces fréquences sont-elles présentes, et comment le taux d'échantillonnage influence-t-il ce que vous observez? (1pt/20)

Écrire votre réponse ici pour d).

On observe dans le spectre les harmoniques du signal d'origine à ± 100 , ± 300 et ± 500 Hz. On observe aussi une composante à ± 450 Hz, qui correspond au parasite réel à 750 Hz replié dans la bande de Nyquist, car la fréquence d'échantillonnage de 1200 Hz ne permet pas de mesurer directement une fréquence supérieure à 600 Hz. Le taux d'échantillonnage limite donc les fréquences observables et provoque l'apparition de fréquences repliées (aliasing).

e) [Code] Pour garantir que ce signal soit propre avant de l'utiliser pour la commande du robot, utiliser un filtre anti-chevauchement d'ordre 5 pour éliminer les fréquences supérieures à 510 Hz. Afficher le signal filtré dans le domaine fréquentiel. (1pt/20)

La fonction `butter` de la bibliothèque `scipy` est utilisée pour créer un filtre passe-bas. Elle génère deux séries de coefficients (`b` et `a`) qui définissent le comportement du filtre. Ensuite, la fonction `filtfilt` applique ce filtre au signal, en atténuant les fréquences supérieures à la fréquence de coupure définie.

```
In [ ]: ## Exemple d'utilisation
from scipy.signal import butter, filtfilt

## Génération des coefficients du filtre
b, a = butter(ordre_du_filtre, fréquence_de_coupure_relative)
```

```
## Application du filtre au signal
signal_filtre = filtfilt(b, a, signal)
```

Après le filtrage, le signal peut être échantillonné à une nouvelle fréquence d'échantillonnage en utilisant la fonction `np.interp`.

```
In [ ]: signal_echantillonne_filtre = np.interp(nouveaux_points_temporels, temps_originaux,
```

```
In [33]: # Écrire votre code ici pour e)

# -- Filtre anti-chevauchement (ordre 5, 510 Hz)

from scipy.signal import butter, filtfilt

# --- Paramètres ---
fs_sensor = 1200          # fréquence d'échantillonnage
fc = 510                  # fréquence de coupure du filtre
ordre = 5                 # ordre du filtre

# On suppose que t_s et x_s existent déjà (résultat du point c)

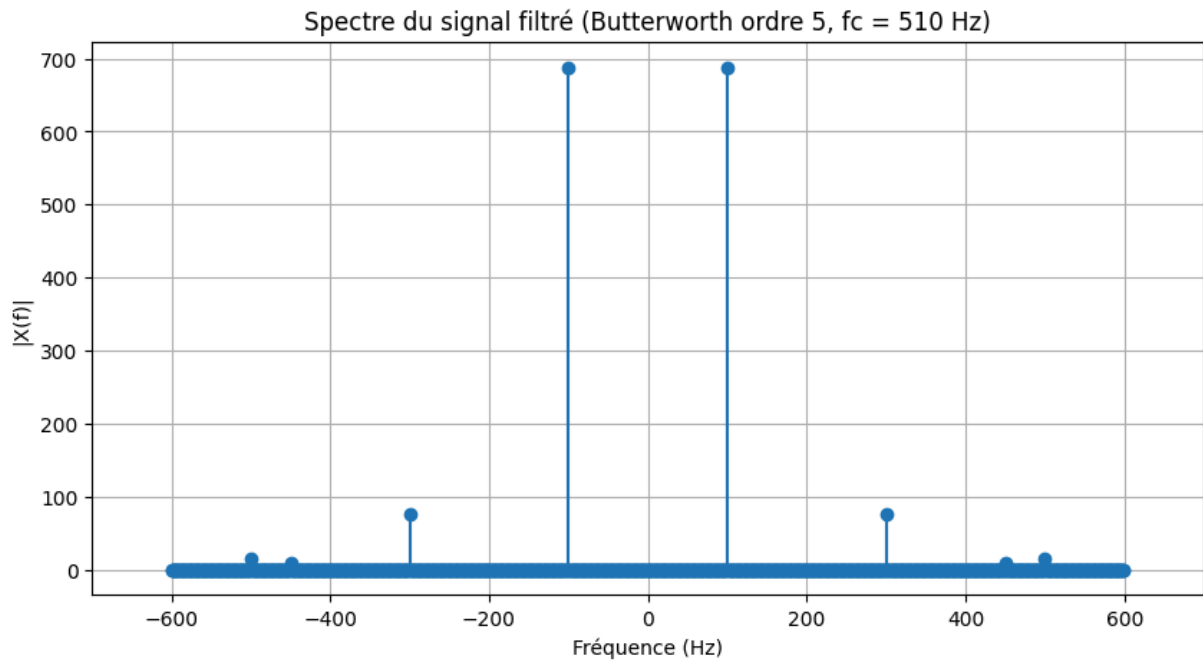
# --- Conception du filtre passe-bas ---
fc_norm_cont = fc / (fs_cont / 2)      # normalisation par rapport à fs_cont (20000)
b, a = butter(ordre, fc_norm_cont, btype='low')

# --- Application du filtre au signal continu ---
x_cont_filtre = filtfilt(b, a, x_cont)

# --- Ré-échantillonnage APRES filtrage (anti-aliasing correct) ---
t_s = generate_time_vec(t1, t2, fs_sensor, endpoint=False)
x_filtre = np.interp(t_s, t_cont, x_cont_filtre)

# --- FFT du signal filtré + échantillonné ---
N = len(x_filtre)
Xf = fft(x_filtre)
freqs = fftfreq(N, d=1/fs_sensor)
Xmag = np.abs(Xf)

# --- Affichage du spectre (miroir) ---
plt.figure(figsize=(10,5))
plt.stem(freqs, Xmag, basefmt=" ")
plt.xlim(-700, 700)
plt.title("Spectre du signal filtré (Butterworth ordre 5, fc = 510 Hz)")
plt.xlabel("Fréquence (Hz)")
plt.ylabel("|X(f)|")
plt.grid(True)
plt.show()
```



f) [Analyse écrite] Quel est l'effet du filtre anti-chevauchement sur le signal échantillonné? Pourquoi est-il essentiel, dans une application robotique, d'utiliser un tel filtre avant de prendre des décisions basées sur l'entrée échantillonnée? (2pt/20)

Écrire votre réponse ici pour f).

Le filtre anti-chevauchement élimine les hautes fréquences avant l'échantillonnage pour éviter qu'elles ne se replient dans la bande utile. Cela empêche l'apparition de fréquences aliasées (comme le 450 Hz dû au parasite à 750 Hz). En robotique, ce filtrage est essentiel car il garantit des mesures fiables : sans cela, le contrôleur réagirait à de fausses informations, ce qui pourrait provoquer des erreurs, des oscillations ou des comportements instables du robot.

Précisions pour la remise:

- Pour chaque exercice, répondre en ajoutant des cases de code ou de texte en dessous de la question.
- Remettez sur moodle un fichier `.ipynb` ainsi qu'une version `pdf` contenant seulement les exercices 2 et 3) avec toutes les librairies et importations nécessaires pour que le code roule.
- Indiquer noms, matricules et numéro d'équipe au début du `notebook`.